ooo xxxx 관제 서버

Naver Cloud Platform 기반 서버 아키텍처 설계서

문서 버전	v1.0.0
작성일	2026-03-09
작성자	UTAREX 양기정 연구원 (kjyang@utarex.com)
기준 문서	요구사항 정의서 v1.0.0
클라우드 플랫폼	Naver Cloud Platform (NCP)
보안 등급	-

목 차

1. 문서 개요 4
1.1 작성 목적 4
1.2 설계 원칙 4
1.3 용어 정의 4
2. 시스템 요구사항 요약 5
2.1 기능 요구사항 요약 5
2.2 비기능 요구사항 5
2.3 데이터 규모 요약 6
3. 전체 시스템 아키텍처 7
3.1 아키텍처 개요 7
3.2 시스템 구성도 (논리 아키텍처) 7
3.3 데이터 흐름 8
4. 소프트웨어 아키텍처 설계 9
4.1 아키텍처 패턴 9
4.1.1 모듈형 모놀리스 → 마이크로서비스 이행 전략 9
4.1.2 레이어드 아키텍처 (Layered Architecture) 9
4.1.3 CQRS 부분 적용 10
4.2 핵심 모듈 구성 10
4.3 기술 스택 10
5. NCP 인프라 설계 11
5.1 네트워크 구성 (VPC) 11
5.2 NCP 서비스 선정 11
5.3 고가용성 구성 12
6. 데이터베이스 설계 13
6.1 DB 계층 전략 13
6.2 주요 테이블 설계 13
6.2.1 핵심 테이블 목록 13
6.2.2 파티셔닝 전략 13
6.3 Redis 캐시 키 설계 14
7. API 설계 15
7.1 데이터 수집 API 15
7.2 조회 API 목록 15
8. 비용 추정 (NCP 기준) 16
8.1 시나리오 A — 초기 운영 (게이트웨이 500대 수준) 16

8.2 시나리오 B — 최대 규모 (게이트웨이 1,000대 풀스케일) 17
8.3 비용 최적화 방안 17
9. 보안 설계 18
9.1 네트워크 보안 18
9.2 인증 18
9.3 데이터 보안 18
9.4 애플리케이션 보안 19
9.5 접근 로그 19
10. 운영 및 배포 전략 20
10.1 배포 파이프라인 20
10.2 모니터링 체계 20
11. 구현 체크리스트 21
11.1 인프라 구성 21
11.2 애플리케이션 개발 21
11.3 보안 22
11.4 성능 검증 22
11.5 운영 준비 23
12. 이행 계획 (단계별 로드맵) 24
13. 문서 변경 이력 24

1. 문서 개요

1.1 작성 목적

본 문서는 XXXX 현장 IoT 게이트웨이로부터 수집되는 센서 및 설비 데이터를 Naver Cloud Platform(NCP) 기반의 독립 관제 서버로 처리하기 위한 서버 아키텍처를 설계한다.

요구사항 정의서(v1.4.1)에 명시된 기능 요건, 데이터 규모 가정, 화면 요구사항을 기반으로 최적의 NCP 서비스 조합과 소프트웨어 아키텍처 패턴을 제시하며, 비용 효율성·구현 난이도·유지보수 용이성을 균형 있게 고려하여 설계한다.

1.2 설계 원칙

- 비용 최적화: NCP 관리형 서비스(Managed Service) 우선 채택으로 운영 부담 최소화
- 확장성: 게이트웨이 500대(초기) → 1,000대(최대) 무중단 수평 확장 지원
- 고가용성: 단일 장애점(SPOF) 제거, 다중 가용 영역(AZ) 구성
- 보안: VPC 내부 망 분리, 최소 권한 원칙 적용
- 유지보수: 마이크로서비스 지향이나 초기 구현은 모듈형 모놀리스로 시작

1.3 용어 정의

용어	정의
NCP	Naver Cloud Platform — 네이버가 제공하는 퍼블릭 클라우드 플랫폼
GW (게이트웨이)	현장 IoT 센서 데이터를 수집·전처리 후 서버로 전송하는 엣지 장치
관제 서버	게이트웨이로부터 데이터를 이중 수집하여 모니터링하는 독립 플랫폼
5분 평균 데이터	게이트웨이 내부에서 5분 단위로 집계된 센서 평균값
30분 집계 데이터	관제 서버에서 5분 데이터를 6개 묶어 생성하는 2차 집계 데이터
VPC	Virtual Private Cloud — 논리적으로 격리된 네트워크 환경
LB	Load Balancer — 트래픽 분산 장치
WAS	Web Application Server — REST API 처리 서버

2. 시스템 요구사항 요약

2.1 기능 요구사항 요약

구분	요구사항	우선순위
데이터 수집	REST API로 게이트웨이 5분 평균 배치 데이터 수신	필수
데이터 저장	5분 데이터 60일 + 30분 집계 데이터 60일 보관	필수
실시간 모니터링	전체 사업장·게이트웨이·센서 상태 조회	필수
이상 탐지	게이트웨이 15분 이상 미수신 시 이상 이벤트 생성	필수
이력 조회	센서 데이터 시계열 그래프 및 이벤트 이력 조회	필수
운영 분석	AI 기반 장애 패턴 분석 및 예측	선택
재시도 처리	통신 실패 시 게이트웨이가 3회 재시도 (지수 백오프) 후 로깅	필수

2.2 비기능 요구사항

항목	목표 수치	근거
데이터 수집 지연	≤ 6분 (게이트웨이 5분 집계 + 서버 처리 1분 이내)	5분 주기 배치 수신
API 응답 시간	≤ 500ms (P95)	대시보드 조회 UX
시스템 가용성	≥ 99.5% (월 허용 장애 약 3.6시간)	관제 시스템 특성
최대 동시 API TPS	≥ 5 TPS (초기) / ≥ 10 TPS (최대)	500대 × 288 req/day / 1,000대 × 288 req/day
저장 용량	≥ 2 TB (초기) / ≥ 4 TB (최대)	60일 기준
보관 기간	60일 (5분 데이터 및 30분 집계 데이터 동일 적용)	요구사항 정의서 v1기준

2.3 데이터 규모 요약

항목	초기 (500대)	최대 (1,000대)
게이트웨이 수	500대	1,000대
게이트웨이당 최대 센서	1,000개	1,000개
전체 일일 레코드 발생량	144,000,000건/일	288,000,000건/일
60일 5분 데이터 총량	8,640,000,000건	17,280,000,000건
30분 집계 데이터 총량 (60일)	1,440,000,000건	2,880,000,000건
총 저장 용량 (60일)	약 1 TB ~ 2 TB	약 2 TB ~ 4 TB
일일 API 요청 수	144,000건/일	288,000건/일
최대 동시 API TPS	≥ 5 TPS	≥ 10 TPS

3. 전체 시스템 아키텍처

3.1 아키텍처 개요

전체 시스템은 데이터 수집 레이어, 메시지 큐 레이어, 처리·저장 레이어, 서비스 레이어, 관제 UI 레이어의 5계층으로 구성된다.

3.2 시스템 구성도 (논리 아키텍처)

순서	구성 요소	네트워크	연결 방식	역할
①	현장 IoT 게이트웨이 × 최대 1,000대	Internet	HTTPS POST /api/v1/gateways/{id}/data	센서 데이터 배치 전송
②	Network Load Balancer (TLS)	Public Subnet	→ Round-Robin	TLS 종단, 트래픽 분산
③	API WAS × N (Auto Scaling Group)	Private Subnet A	→ Kafka Produce	REST API 수신 / 유효성 검증
④	Cloud Kafka (3 Broker / RF=2)	Private Subnet B	→ Kafka Consume	비동기 버퍼링 / 역압 처리
⑤	Consumer Worker × N (Auto Scaling)	Private Subnet A	→ INSERT / READ	데이터 저장 / 집계 / 이상탐지
⑥	MySQL M+S / Redis / Object Storage	Private Subnet B	← READ (Cache-Aside)	Hot Data / 캐시 / Cold 아카이브
⑦	조회 API WAS × N (Auto Scaling)	Private Subnet A	→ HTTPS	대시보드 / 현장 상세 데이터 제공
⑧	관제 UI — CDN + Object Storage (SPA)	Internet	→ 관리자 브라우저	웹 관제 화면 서비스

3.3 데이터 흐름

단계	흐름	처리 내용
1단계	GW → Load Balancer	HTTPS POST /api/v1/gateways/{gatewayId}/data 호출 (배치, 최대 1,000건)
2단계	Load Balancer → API WAS	TLS 해제 후 API 서버 라운드로빈 분산
3단계	API WAS → Kafka	수신 즉시 Kafka 토픽 발행 (ack 후 즉시 응답 200)
4단계	Kafka → Consumer Worker	5분 데이터 DB 저장, 30분 집계 생성, 이상 탐지
5단계	DB/Redis → 조회 API → UI	대시보드·현장 상세 화면 데이터 제공

4. 소프트웨어 아키텍처 설계

4.1 아키텍처 패턴

4.1.1 모듈형 모놀리스 → 마이크로서비스 이행 전략

초기에는 단일 배포 단위(모놀리스)로 시작하되 내부를 명확한 모듈 경계(도메인 패키지)로 분리하여 유지보수성을 확보한다. 게이트웨이 규모가 500대 이상으로 확대되거나 팀 규모가 성장하는 시점에 특정 모듈을 마이크로서비스로 분리 추출하는 이행 경로를 제시한다.

기준	모듈형 모놀리스 (초기 선택)	마이크로서비스 (확장 후)
구현 난이도	낮음	높음
운영 복잡도	낮음	높음
확장성	모듈 단위 수평 확장 가능	서비스 단위 독립 확장
배포 단위	단일 Docker 컨테이너	서비스별 독립 배포
초기 비용	낮음	높음

4.1.2 레이어드 아키텍처 (Layered Architecture)

애플리케이션 내부는 4개 레이어로 분리하여 역할을 명확히 한다.

레이어	구성 요소	책임
Presentation Layer	REST Controller, DTO	HTTP 요청/응답 처리, 입력 유효성 검증
Application Layer	Service, Use Case	비즈니스 로직 조율, 트랜잭션 관리
Domain Layer	Entity, Domain Service, Repository Interface	핵심 비즈니스 규칙, 도메인 모델
Infrastructure Layer	Repository Impl, Kafka Producer/Consumer, DB Adapter	외부 시스템 연동 구현체

4.1.3 CQRS 부분 적용

데이터 수집(Write)과 대시보드 조회(Read)는 부하 특성이 완전히 다르다. 수집은 초당 최대 수천 건의 INSERT 위주이며, 조회는 복잡한 JOIN·집계 위주다. 수집 경로와 조회 경로를 분리하고 조회용 Read Model은 Redis 캐시로 사전 계산하여 제공한다.

4.2 핵심 모듈 구성

모듈명	역할	주요 기술
data-ingest	게이트웨이 데이터 수신 REST API	Express.js, kafkajs (Producer)
data-processor	Kafka 소비·저장·집계·이상탐지	kafkajs (Consumer), node-cron
monitoring-api	대시보드·현장 상세 조회 API	Express.js, ioredis, mysql2
alarm-engine	이상 상태 감지·알림 발송	node-cron, NCP SENS
common	공통 유틸, 예외 처리, 미들웨어	helmet, express-rate-limit

4.3 기술 스택

분류	기술	선택 이유
언어	Node.js 22 (LTS)	경량·빠른 I/O 처리, 풍부한 생태계
프레임워크	Express.js	구조 단순, 러닝커브 낮음, NCP SDK 호환
DB 드라이버	mysql2	Promise 지원, 빠른 MySQL 연결
캐시 클라이언트	ioredis	Redis 공식 Node.js 클라이언트
메시지 큐	NCP Cloud Kafka + kafkajs	관리형 Kafka, Node.js 네이티브 클라이언트
스케줄러	node-cron	30분 집계·파티션 관리 등 주기 작업
보안 미들웨어	helmet + express-rate-limit	보안 헤더 자동 설정, 요청 횟수 제한
컨테이너	Docker + NCP Container Registry	환경 일관성 보장, 이미지 프라이빗 저장
CI/CD	NCP SourcePipeline	NCP 네이티브 빌드·배포 통합
모니터링	NCP Cloud Insight + Log Analytics	메트릭·로그·알림 통합

5. NCP 인프라 설계

5.1 네트워크 구성 (VPC)

서브넷	용도	CIDR 예시	접근 범위
Public Subnet	Load Balancer, Bastion Host	10.0.1.0/24	인터넷 허용
Private Subnet A	API WAS, Consumer Worker	10.0.2.0/24	내부망 전용
Private Subnet B	DB, Redis, Kafka	10.0.3.0/24	내부망 전용
Private Subnet C	관리·모니터링	10.0.4.0/24	내부망 전용

5.2 NCP 서비스 선정

계층	NCP 서비스	사양 (초기/최대)	선택 이유
로드밸런서	Network Load Balancer	Standard / Standard	L4 수준, 고성능, TLS 패스스루
API 서버	Server (Auto Scaling)	c2-g3 2vCPU/4GB × 2대 → 최대 10대	비용 효율, 자동 수평 확장
Consumer	Server (Auto Scaling)	c2-g3 2vCPU/4GB × 2대 → 최대 8대	Kafka 소비 병렬 처리
메시지 큐	Cloud Kafka	2 Broker → 3 Broker (Phase 2 증설 권장)	관리형, 고가용성
관계형 DB	Cloud DB for MySQL	db.c2.c4m8 M+S → M+2S	자동 페일오버, 자동 백업
캐시	Cloud DB for Redis	Cache-8GB → HA Cluster	대시보드 빠른 응답
오브젝트 스토리지	Object Storage	초기 2TB → 10TB+	저비용 장기 보관, S3 호환

계층	NCP 서비스	사양 (초기/최대)	선택 이유
컨테이너	NKS (Kubernetes)	선택 사항 (중기 도입)	무중단 배포 자동화
모니터링	Cloud Insight + Log Analytics	기본 플랜	통합 모니터링
알림	SENS	메일 + SMS	이상 이벤트 알림

5.3 고가용성 구성

- Load Balancer: Active-Standby 이중화 (NCP 관리형)
- API WAS: Auto Scaling Group, 최소 2대 → Health Check 기반 자동 교체
- Cloud DB for MySQL: Master + Slave 구성, 자동 페일오버 (장애 시 Slave 자동 승격)
- Cloud Kafka: 초기 2 Broker 구성은 고가용성 미보장 상태이며, Phase 2 이전에 3 Broker + Replication Factor 2로 증설하여 브로커 1대 장애 허용 구성으로 전환 필요
- Redis: 초기 Single → 중기 이후 Sentinel 또는 Cluster 전환

6. 데이터베이스 설계

6.1 DB 계층 전략

계층	저장소	저장 데이터	보관 기간
Hot Data	Cloud DB for MySQL	최근 60일 5분 데이터, 30분 집계 데이터(60일), 이벤트	60일 (5분 및 30분 집계 동일)
Warm Cache	Cloud DB for Redis	대시보드 집계, 최근 상태, 세션	수분~수시간
Cold Data	Object Storage	60일 이후 아카이브 (Parquet/CSV)	무제한

6.2 주요 테이블 설계

6.2.1 핵심 테이블 목록

테이블명	설명	예상 레코드 수 (60일)
sites	사업장 마스터	~ 수백 건
gateways	게이트웨이 마스터	최대 1,000건
sensors	센서 채널 마스터	최대 1,000,000건
sensor_data_5m	5분 평균 센서 데이터 (파티셔닝)	최대 172.8억 건
sensor_data_30m	30분 집계 센서 데이터 (60일 보관)	최대 28.8억 건
gateway_status	게이트웨이 상태 이력	최대 1,728만 건
alarm_events	이상 이벤트 이력	이벤트 발생 기준

6.2.2 파티셔닝 전략

sensor_data_5m 테이블은 일 최대 2.88억 건이 INSERT되므로 반드시 파티셔닝이 필요하다.

- 파티셔닝 방식: RANGE PARTITION by collected_at (일 단위) — 60일치 파티션 생성 후 롤링 방식으로 삭제

- 일 단위 파티션 자동 생성: 매일 00:00 node-cron 스케줄러 실행
- 만료 파티션 자동 삭제: 61일째 파티션 DROP (ALTER TABLE DROP PARTITION)
- 인덱스: (gateway_id, collected_at), (sensor_id, collected_at) 복합 인덱스

6.3 Redis 캐시 키 설계

캐시 키 패턴	TTL	내용
dashboard:summary	30초	전체 사업장 요약 (총 수, 정상/이상, 미수신)
site:list:page:{n}	30초	현장 목록 페이지 캐시
site:detail:{siteId}	10초	현장 상세 최신 상태
gw:status:{gwId}	5초	게이트웨이 최신 상태
sensor:latest:{sensorId}	60초	센서 최신 측정값

7. API 설계

7.1 데이터 수집 API

항목	내용
Endpoint	POST /api/v1/gateways/{gatewayId}/data
인증	API Key (헤더: X-API-Key)
요청 형식	JSON Array (배치, 최대 1,000건)
응답 형식	{"status": "accepted", "count": N}
처리 방식	수신 즉시 Kafka 발행 → 비동기 처리 (빠른 응답)
재시도 정책	통신 실패 시 GW 측 3회 재시도 (지수 백오프)

7.2 조회 API 목록

API	Method	Endpoint	캐시
통합 대시보드	GET	/api/v1/dashboard/summary	Redis 30s
현장 목록	GET	/api/v1/sites?page={n}&size={s}	Redis 30s
현장 상세	GET	/api/v1/sites/{siteId}	Redis 10s
게이트웨이 상태	GET	/api/v1/gateways/{gwId}/status	Redis 5s
센서 시계열	GET	/api/v1/sensors/{sId}/timeseries?from={}&to={}	DB 직접
이상 이벤트	GET	/api/v1/sites/{siteId}/alarms	DB 직접
가동률 조회	GET	/api/v1/sites/{siteId}/availability	Redis 60s

8. 비용 추정 (NCP 기준)

아래 비용은 NCP 공개 단가 기준 추정값이며, 계약 할인·Reserved Instance 적용 시 감소할 수 있다. 부가세(10%) 미포함.

8.1 시나리오 A — 초기 운영 (게이트웨이 500대 수준)

서비스	사양	수량	월 예상 비용(원)
Network Load Balancer	Standard	1	약 50,000
Server (API WAS)	c2-g3 2vCPU/4GB	3대	약 180,000
Server (Consumer)	c2-g3 2vCPU/4GB	3대	약 180,000
Cloud Kafka	2 Broker, 500GB	1 클러스터	약 350,000
Cloud DB for MySQL	db.c2.c4m8 M+S, 2TB	1 세트	약 600,000
Cloud DB for Redis	Cache-8GB	1	약 100,000
Object Storage	2TB	–	약 50,000
Cloud Insight	기본 플랜	–	약 30,000
SENS (알림)	메일+SMS 기본	–	약 20,000
기타 (네트워크 트래픽)	–	–	약 70,000
합계 (월)	–	–	**약 1,630,000원/월**

8.2 시나리오 B — 최대 규모 (게이트웨이 1,000대 풀스케일)

서비스	사양	수량	월 예상 비용(원)
Network Load Balancer	Standard × 2 (이중화)	2	약 100,000
Server (API WAS + Auto Scaling)	c2-g3 ~ c4-g3, 최대 10대	평균 6대	약 500,000
Server (Consumer + Auto Scaling)	c2-g3 ~ c4-g3, 최대 8대	평균 4대	약 300,000
Cloud Kafka	3 Broker, 1TB	1 클러스터	약 600,000
Cloud DB for MySQL	db.c4.c8m16 M+2S, 5TB	1 세트	약 1,800,000
Cloud DB for Redis	Cache-32GB HA Cluster	1 클러스터	약 400,000
Object Storage	10TB+	–	약 200,000
NKS (Kubernetes)	기본 클러스터	1	약 300,000
Cloud Insight + Log Analytics	확장 플랜	–	약 150,000
SENS (알림)	확장 플랜	–	약 50,000
기타 (네트워크, CDN 등)	–	–	약 200,000
합계 (월)	–	–	**약 4,600,000원/월**

8.3 비용 최적화 방안

- Reserved Instance 약정 (1년): 최대 30~40% 할인
- Auto Scaling: 야간·주말 최소 인스턴스로 자동 축소
- Cold 데이터 Object Storage 이관: MySQL 60일 초과 데이터 Parquet 변환 후 삭제
- DB 파티션 롤링 삭제로 MySQL 용량 증가 억제
- Redis TTL 최적화: 조회 빈도 낮은 데이터 TTL 연장

9. 보안 설계

9.1 네트워크 보안

항목	적용 내용
VPC 망 분리	Public / Private Subnet 분리 — DB·Kafka·Redis는 외부에서 직접 접근 불가
ACG 포트 제어	허용 포트 최소화 — 인터넷→LB: 443만 허용, 서버간 통신은 해당 포트만 개방
Bastion Host	SSH 접근은 Bastion Host 경유만 허용, MFA(다중 인증) 적용

9.2 인증

대상	인증 방식	비고
게이트웨이 (데이터 수집)	API Key — X-API-Key 헤더 검증	게이트웨이 1대당 Key 1개 발급, 유출 시 즉시 폐기·재발급
관리자 (관제 UI)	JWT — HS256 서명, 유효 시간 8시간	만료 후 재로그인, 관리자 접근 IP 화이트리스트 적용

9.3 데이터 보안

항목	적용 내용
전송 암호화	TLS 1.2 이상 — 게이트웨이↔서버, 브라우저↔서버 전 구간 HTTPS 적용
저장 암호화	API Key·관리자 비밀번호: bcrypt 단방향 해시 저장 / Object Storage: SSE 적용
시크릿 관리	DB 접속 정보·JWT 키 등 민감 설정값은 환경 변수로 관리, 소스 코드 및 Git 저장소 포함 금지

항목	적용 내용
DB 계정 분리	수집용(INSERT/UPDATE) · 조회용(SELECT) 계정 분리 — 최소 권한 원칙 적용

9.4 애플리케이션 보안

항목	적용 기술	효과
보안 헤더	helmet (Node.js 라이브러리)	XSS·클릭재킹 등 일반 웹 공격 방어
요청 횟수 제한	express-rate-limit	브루트포스 공격 및 비정상 대량 요청 차단
컨테이너 보안	Docker non-root 실행 + alpine 이미지	공격 표면 최소화, 컨테이너 탈출 시 피해 제한
취약점 점검	npm audit + NCP Web Security Checker	의존성 패키지 및 웹 취약점 정기 점검

9.5 접근 로그

로그는 NCP Log Analytics로 중앙 수집되며, 단시간 다수 인증 실패·비정상 IP 접근 등 이상 패턴 감지 시 관리자에게 즉시 알림을 발송한다.

로그 종류	기록 내용	보관
API 접근 로그	요청 시각, 출발지 IP, API Key, 응답 코드	90일
인증 실패 로그	실패 시각, IP, 사용된 계정 또는 Key	1년
관리자 로그인 로그	시각, IP, 성공/실패 여부	1년

10. 운영 및 배포 전략

10.1 배포 파이프라인

단계	도구	내용
소스 관리	GitHub / NCP SourceCommit	Git Flow 브랜치 전략 (main/develop/feature)
빌드	NCP SourceBuild	npm install·테스트 실행, Docker 이미지 빌드
이미지 저장	NCP Container Registry	Docker 이미지 프라이빗 저장 (외부 공개 차단)
배포	NCP SourceDeploy	Blue-Green 배포 (무중단), 자동 롤백
인프라 관리	Terraform	NCP 리소스 IaC 관리

10.2 모니터링 체계

모니터링 항목	도구	임계값 / 알림 조건
서버 CPU/메모리	Cloud Insight	CPU 80% 이상 5분 지속 → 알림
API 응답 시간	Cloud Insight + prom-client	P95 > 1초 → 알림
Kafka 소비 지연	Cloud Insight (Consumer Lag)	Lag > 10,000건 → 알림
DB 연결 수	Cloud DB Insight	연결 수 80% 이상 → 알림
데이터 미수신 GW	자체 스케줄러 (15분 기준)	15분 이상 미수신 GW → 이상 이벤트 생성
디스크 사용량	Cloud Insight	80% 이상 → 알림
애플리케이션 에러	Log Analytics	ERROR 레벨 로그 발생 → 즉시 알림

11. 구현 체크리스트

11.1 인프라 구성

#	항목	확인
1	NCP VPC 생성 및 서브넷(Public/Private) 분리 구성	☐
2	ACG 규칙 설정 (LB→WAS:8080, WAS→DB:3306, WAS→Redis:6379, WAS→Kafka:9092)	☐
3	Network Load Balancer 생성 및 TLS 인증서 적용	☐
4	Cloud Kafka 클러스터 생성, 토픽(sensor-data-raw) 파티션 수 설정	☐
5	Cloud DB for MySQL 생성, Master+Slave 복제 확인	☐
6	Cloud DB for Redis 생성, 연결 테스트	☐
7	Object Storage 버킷 생성, 접근 정책(비공개) 설정	☐
8	Bastion Host 생성, SSH 키 페어 관리	☐
9	Auto Scaling Group 생성, Scale-out/in 정책 설정	☐
10	Cloud Insight 대시보드 및 알림 설정	☐

11.2 애플리케이션 개발

#	항목	확인
1	데이터 수집 API 구현 (POST /api/v1/gateways/{id}/data)	☐
2	API Key 인증 미들웨어 구현	☐
3	Kafka Producer 구현 (수신 데이터 즉시 발행)	☐
4	Kafka Consumer 구현 (DB 저장, 이상 탐지 트리거)	☐
5	sensor_data_5m 테이블 파티셔닝 설정 및 자동 관리 스케줄러	☐
6	30분 집계 배치 구현 (node-cron 스케줄러)	☐

#	항목	확인
7	이상 탐지 로직 구현 (게이트웨이 미수신 15분 기준)	☐
8	통합 대시보드 조회 API + Redis 캐싱 구현	☐
9	현장 목록 / 현장 상세 조회 API 구현	☐
10	센서 시계열 조회 API 구현 (from/to 범위 조회)	☐
11	이상 이벤트 이력 조회 API 구현	☐
12	Object Storage Cold 이관 배치 구현 (61일 이상 데이터)	☐
13	API 요청/응답 로깅 필터 구현	☐
14	Graceful Shutdown 구현 (Kafka Consumer 안전 종료)	☐

11.3 보안

#	항목	확인
1	모든 외부 통신 HTTPS 적용 확인	☐
2	API Key 발급·관리 정책 수립	☐
3	DB 접속 계정 최소 권한 분리 (read-only / write 계정 분리)	☐
4	환경 변수·시크릿 하드코딩 금지 (NCP Secret Manager 또는 환경 변수 관리)	☐
5	VPC ACG 최소 허용 원칙 검토	☐
6	관리자 UI 접근 IP 화이트리스트 설정	☐

11.4 성능 검증

#	항목	확인
1	최대 부하 시나리오 부하 테스트 (nGrinder 또는 k6 활용)	☐
2	대용량 INSERT 성능 테스트 (배치 INSERT 1,000건/요청)	☐
3	Redis 캐시 Hit Rate 모니터링 설정 (목표: 80% 이상)	☐
4	DB 슬로우 쿼리 로그 활성화 및 임계값 설정 (> 1초)	☐

#	항목	확인
5	Kafka Consumer Lag 모니터링 및 Scale-out 트리거 검증	☐
6	파티션 삭제 배치 영향도 테스트 (운영 중 DROP PARTITION)	☐

11.5 운영 준비

#	항목	확인
1	CI/CD 파이프라인 구성 및 Blue-Green 배포 검증	☐
2	DB 자동 백업 설정 확인 (Cloud DB 일일 자동 백업)	☐
3	장애 복구 훈련 (DR Drill): DB Failover 시나리오	☐
4	운영 매뉴얼 작성 (장애 대응 Runbook)	☐
5	모니터링 알림 수신자 등록 및 에스컬레이션 정책 수립	☐
6	데이터 보관 정책 문서화 및 자동화 스크립트 검증	☐

12. 이행 계획 (단계별 로드맵)

단계	기간	주요 작업	목표 규모
Phase 1 (MVP)	1~2개월	VPC·DB·Kafka 기본 구성, 수집 API + Consumer 구현, 통합 대시보드 기본 화면	~500 GW
Phase 2 (안정화)	3~4개월	30분 집계 배치 구현, 이상 탐지·알림 구현, 현장 상세·이력 조회, 부하 테스트·튜닝, Kafka 3 Broker 증설	~800 GW
Phase 3 (확장)	5~6개월	Auto Scaling 정책 최적화, NKS 도입 검토, Cold 이관 자동화, AI 기반 장애 패턴 분석 및 예측 기능 추가	~1,000 GW

13. 문서 변경 이력

버전	일자	작성자	변경 내용
v1.0.0	2026-03-09	양기정	최초 작성